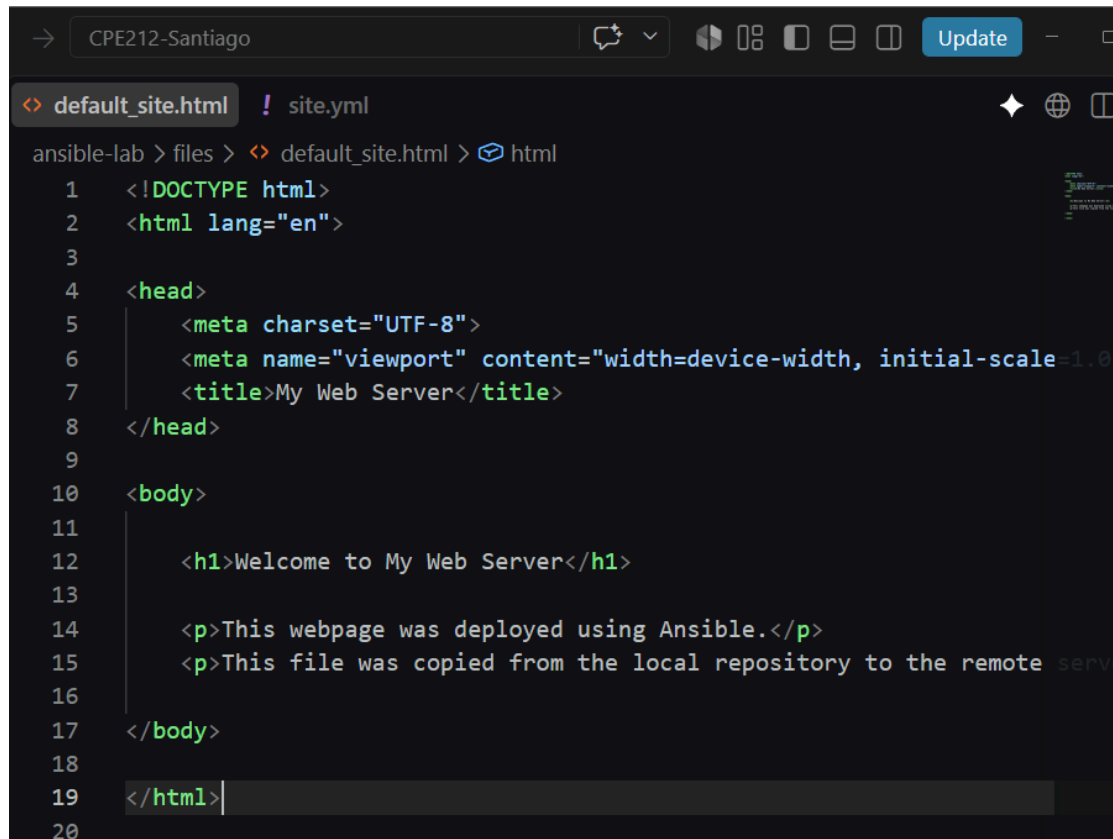


COURSE CODE/TITLE:	CPE 212 - Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	09 - 10 - 26	DATE SUBMITTED:	09 - 13 - 26
NAME:	Santiago, David Owen A.	INSTRUCTOR:	Engr. Jimlord Quejado

Hands-On Activity 7.1	Managing Files and Creating Roles in Ansible
1. PROCEDURE OUTPUT	
Task 1: Create a file and copy it to remote servers 1. Using the previous directory that was created in your previous laboratory, create a directory, and name it “<i>files</i>.” Create a file inside that directory and name it “<i>default_site.html</i>.” Edit the file and put basic HTML syntax. Any content will do, as long as it will display text later. Save the file and exit. Example: <pre><!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>My Web Server</title> </head> <body> <h1>Welcome to My Web Server</h1> <p>This webpage was deployed using Ansible.</p> <p>This file was copied from the local repository to the remote server.</p> </body> </html></pre> OUTPUT:	



```

CPE212-Santiago
default_site.html ! site.yml
ansible-lab > files > default_site.html > html
1  <!DOCTYPE html>
2  <html lang="en">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>My Web Server</title>
8  </head>
9
10 <body>
11
12     <h1>Welcome to My Web Server</h1>
13
14     <p>This webpage was deployed using Ansible.</p>
15     <p>This file was copied from the local repository to the remote server.</p>
16
17 </body>
18
19 </html>
20

```

2. Edit the `site.yml` file and just below the `web_servers` play, create a new file to copy the default html file for site:

```

- name: copy default html file for
  site  tags: apache, apache2, httpd
  copy:
    src: files/default_site.html
    dest: /var/www/html/index.html
    owner: root
    group: root
    mode: '0644'

```

OUTPUT:

```
- hosts: web_servers
  become: true
  roles:
    - role: web
      tags: web
  tasks:
    - name: copy default html file for site
      tags: apache, apache2, httpd
      ansible.builtin.copy:
        src: files/default_site.html
        dest: /var/www/html/index.html
        owner: root
        group: root
        mode: '0644'
```

3. Run the playbook *site.yml*. Describe the changes.

```
TASK [copy default html file for site] *****
changed: [ubuntu1]
changed: [centos1]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [ubuntu2]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [centos1]

PLAY RECAP *****
centos1                : ok=7    changed=1    unreachable=0    failed=0    skipped=2    rescued=0
  ignored=0
ubuntu1                : ok=5    changed=2    unreachable=0    failed=0    skipped=3    rescued=0
  ignored=0
ubuntu2                : ok=3    changed=1    unreachable=0    failed=0    skipped=1    rescued=0
  ignored=0

santiago@workstation:~/CPE212-Santiago/ansible-lab$
```

My *web_servers* role has both an *ubuntu* and *centos* machine, hence, they both received a change from the copy task. The task executed successfully, even if a task was explicitly stated in the *site.yml* file instead of the *main.yml* file of the role.

4. Go to the remote servers (*web_servers*) listed in your inventory. Use *cat* command to check if the *index.html* is the same as the local repository file (*default_site.html*). Do both for Ubuntu and CentOS servers. On the CentOS server, go to the browser and type its IP address. Describe the output.

```
server1@server1:/var/www/html$ ls
index.html
server1@server1:/var/www/html$ cat index.html
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Web Server</title>
</head>

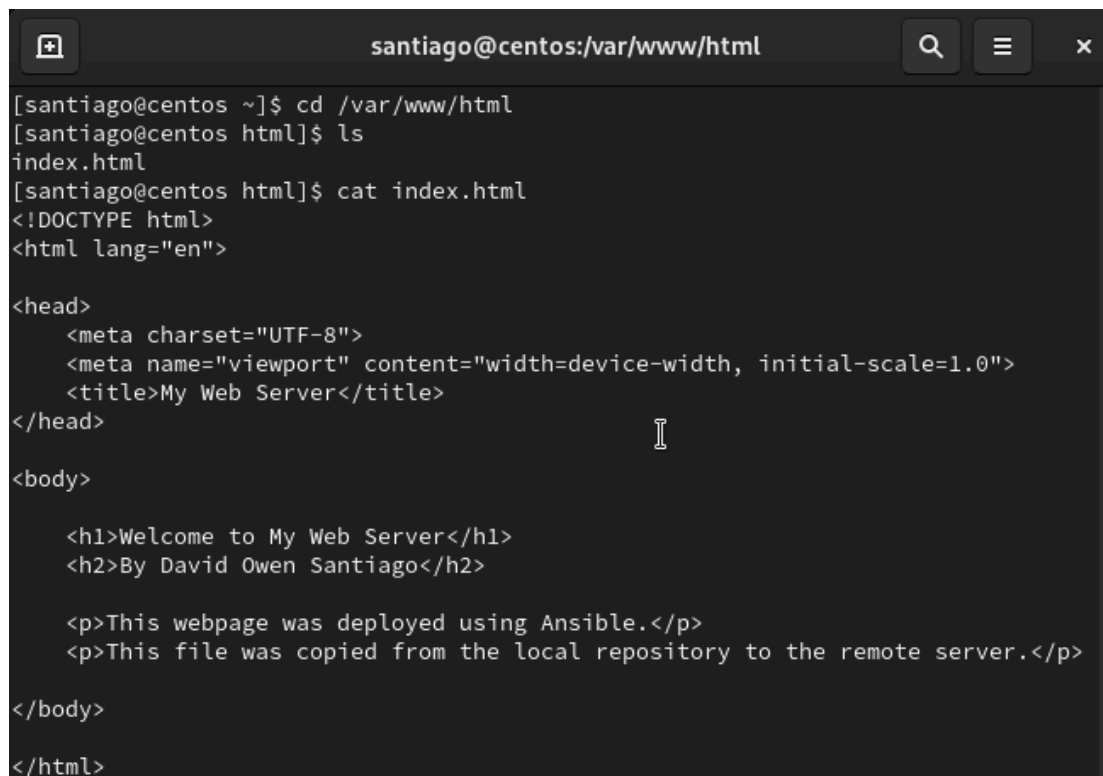
<body>

  <h1>Welcome to My Web Server</h1>
  <h2>By David Owen Santiago</h2>

  <p>This webpage was deployed using Ansible.</p>
  <p>This file was copied from the local repository to the remote server.</p>

</body>

</html>
server1@server1:/var/www/html$
```



```
santiago@centos:/var/www/html
[santiago@centos ~]$ cd /var/www/html
[santiago@centos html]$ ls
index.html
[santiago@centos html]$ cat index.html
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Web Server</title>
</head>

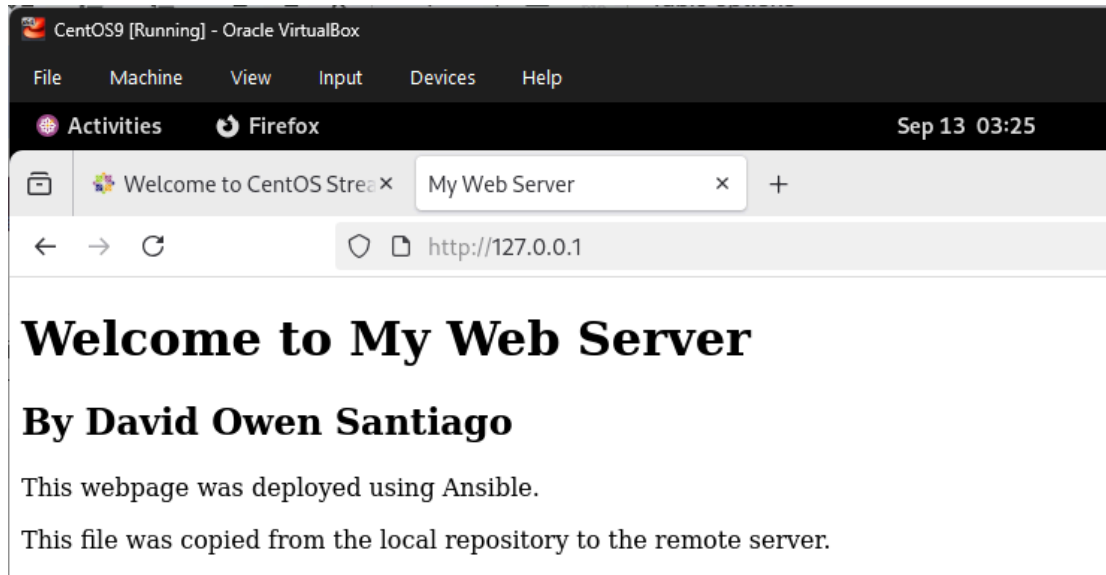
<body>

  <h1>Welcome to My Web Server</h1>
  <h2>By David Owen Santiago</h2>

  <p>This webpage was deployed using Ansible.</p>
  <p>This file was copied from the local repository to the remote server.</p>

</body>

</html>
```



The play successfully copied the index.html file inside all of the web_servers. In CentOS, opening the browser and typing the localhost IP address automatically loads the default webpage containing the contents of the index.html file.

5. Sync your local repository with GitHub and describe the changes.
The changes so far are the new copy tasks for the web_servers role as well as the new files directory for the new default site.html file and future files.

Task 2: Download a file and extract it to a remote server

1. Edit the site.yml. Just before the web_servers play, create a new play:

```
---
- hosts: workstations
  become: true
  tasks:
    - name: install unzip
      package:
        name: unzip

    - name: install terraform
      unarchive:
src:
https://releases.hashicorp.com/terraform/0.12.28/terraform_0.12.28_linux
_amd64.zip
      dest: /usr/local/bin
      remote_src: yes
```

```
mode: 0755
owner: root
group: root
```

Note: In Ansible, a play is a section of a playbook that defines which remote machines Ansible should manage and what tasks it should perform on them.

```
20
21 - hosts: workstations
22   become: true
23   tasks:
24     - name: install unzip
25       package:
26         name: unzip
27
28     - name: install terraform
29       unarchive:
30         src: https://releases.hashicorp.com/terraform/0.12.28/terraform_0.12.28_linux_amd64.zip
31         dest: /usr/local/bin
32         remote_src: yes
33         mode: 0755
34         owner: root
35         group: root
```

2. Edit the inventory file and add a workstation group. Add any Ubuntu remote server. Make sure to remember the IP address.

```
workstations:
  hosts:
    ubuntu1:
      ansible_host: 192.168.56.102
      ansible_user: server1
```

3. Run the playbook. Describe the output.

```
TASK [install terraform] *****
changed: [ubuntu1]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [ubuntu1]
ok: [centos1]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [ubuntu2]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [centos1]

PLAY RECAP *****
centos1                : ok=4   changed=0    unreachable=0    failed=0    skipped=1    rescued=0
ignored=0
ubuntu1                : ok=6   changed=2    unreachable=0    failed=0    skipped=1    rescued=0
ignored=0
ubuntu2                : ok=3   changed=1    unreachable=0    failed=0    skipped=1    rescued=0
ignored=0

santiago@workstation:~/CPE212-Santiago/ansible-lab$
```

Running the playbook executed the workstation task on my specified workstation.

4. On the Ubuntu remote workstation, type terraform to verify installation of terraform. Describe the output.

```
server1@server1:~$ terraform
Usage: terraform [-version] [-help] <command> [args]

The available commands for execution are listed below.
The most common, useful commands are shown first, followed by
less common or more advanced commands. If you're just getting
started with Terraform, stick with the common commands. For the
other commands, please read the help and docs before usage.
```

I successfully verified that terraform was installed in my workstation remote server.

Task 3: Create roles

1. Edit the site.yml. Configure roles as follows: (make sure to create a copy of the old site.yml file because you will be copying the specific plays for all groups)

```
--  
- hosts: all  
  become: true  
  pre_tasks:  
  
    - name: update repository index (CentOS)  
      tags: always  
      dnf:  
        update_cache: yes  
        changed_when: false  
        when: ansible_distribution == "CentOS"  
    - name: install updates (Ubuntu)  
      tags: always  
      apt:  
        update_cache: yes  
        changed_when: false  
        when: ansible_distribution == "Ubuntu"  
  
- hosts: all  
  become: true  
  roles:  
    - base  
  
- hosts: workstations  
  become: true  
  roles:  
    - workstations  
  
- hosts: web_servers  
  become: true  
  roles:  
    - web_servers  
  
- hosts: db_servers  
  become: true  
  roles:  
    - db_servers  
  
- hosts: file_servers  
  become: true  
  roles:  
    - file_servers
```

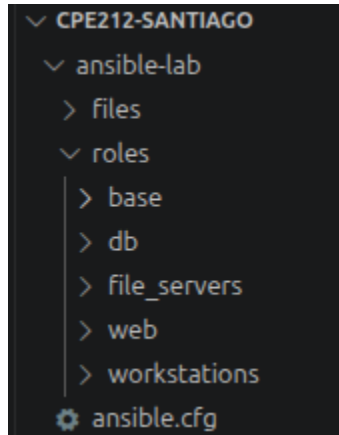
Save the file and exit.

```

! site.yml M X ! main.yml ../file_servers/... U ! main.yml ../w
ansible-lab > ! site.yml
1  ---
2  - hosts: all
3    become: true
4    pre_tasks:
5
6      - name: install updates (CentOS)
7        tags: always
8        dnf:
9          update_only: yes
10         update_cache: yes
11        when: ansible_distribution == "CentOS"
12
13     - name: install updates (Ubuntu)
14       tags: always
15       apt:
16         #upgrade: dist
17         update_cache: yes
18       when: ansible_distribution == "Ubuntu"
19
20  - hosts: workstations
21    become: true
22    roles:
23      - role: workstations
24      tags: workstation
25
26  - hosts: web_servers
27    become: true
28    roles:
29      - role: web
30      tags: web
31
32  - hosts: db_servers
33    become: true
34    roles:
35      - role: db
36      tags: db
37
38  - hosts: file_servers
39    become: true
40    roles:
41      - role: file_servers
42      tags: file_server

```

2. Under the same directory, create a new directory and name it roles. Enter the roles directory and create new directories: base, web_servers, file_servers, db_servers and workstations. For each directory, create a directory and name it tasks.



3. Go to tasks for all directory and create a file. Name it main.yml. In each of the tasks for all directories, copy and paste the code from the old site.yml file. Show all contents of main.yml files for all tasks.

```
ansible-lab > roles > db > tasks > ! main.yml
 1  #SPDX-License-Identifier: MIT-0
 2  ---
 3  # tasks file for db
 4
 5  - name: install mariadb package (CentOS)
 6    tags: centos, db, mariadb
 7    dnf:
 8      name: mariadb-server
 9      state: latest
10    when: ansible_distribution == "CentOS"
11
12  - name: install mariadb package (Ubuntu)
13    tags: db, mariadb, ubuntu
14    apt:
15      name: mariadb-server
16      state: latest
17    when: ansible_distribution == "Ubuntu"
18
19  - name: "Mariadb- Restarting/Enabling"
20    service:
21      name: mariadb
22      state: restarted
23      enabled: true
24
25  - name: "Print Confirm if Successful"
26    debug:
27      msg: "MariaDB is installed and running successfully."
```

```
ansible-lab > roles > file_servers > tasks > ! main.yml
 1  #SPDX-License-Identifier: MIT-0
 2  ---
 3  # tasks file for file_servers
 4  - name: install samba package
 5    tags: samba
 6    package:
 7      name: samba
 8      state: latest
```

```
ansible-lab > roles > web > tasks > ! main.yml
1  #SPDX-License-Identifier: MIT-0
2  ---
3  # tasks file for web
4
5  - name: Install Apache2 package and PHP Support Ubuntu
6    tags: apache, apache2, ubuntu
7    apt:
8      name:
9        - apache2
10       - libapache2-mod-php
11      state: latest
12      update_cache: yes
13      when: ansible_distribution == "Ubuntu" # for Ubuntu
14
15  - name: Install Apache2 package and PHP Support CentOS
16    tags: apache, centos, httpd
17    dnf:
18      name:
19        - httpd
20        - php
21      state: latest
22      update_cache: yes
23      when: ansible_distribution == "CentOS" # for CentOS
24
25  - name: start httpd (CentOS)
26    tags: apache, centos, httpd
27    service:
28      name: httpd
29      state: started
30      enabled: true
31      when: ansible_distribution == "CentOS"
32
33  - name: "Print Confirm if Successful"
34    debug:
35      msg: "Apache and PHP are installed and running successfully."
36
37  - name: copy default html file for site
38    tags: apache, apache2, httpd, copy_html
39    ansible.builtin.copy:
40      src: files/default_site.html
41      dest: /var/www/html/index.html
42      owner: root
43      group: root
44      mode: '0644'
```

```

ansible-lab > roles > workstations > tasks > ! main.yml
1  #SPDX-License-Identifier: MIT-0
2  ---
3  # tasks file for workstations
4  - name: install unzip
5    package:
6      name: unzip
7
8  - name: install terraform
9    unarchive:
10     src: https://releases.hashicorp.com/terraform/0.12.28/terraform_0.12.28_linux_amd64.zip
11     dest: /usr/local/bin
12     remote_src: yes
13     mode: 0755
14     owner: root
15     group: root

```

4. Run the site.yml playbook and describe the output.

```

PLAY RECAP *****
centos1                : ok=9    changed=0    unreachable=0    failed=0    skipped=2    rescued=0
ignored=0
ubuntu1                : ok=9    changed=1    unreachable=0    failed=0    skipped=3    rescued=0
ignored=0
ubuntu2                : ok=6    changed=2    unreachable=0    failed=0    skipped=2    rescued=0
ignored=0

santiago@workstation:~/CPE212-Santiago/ansible-lab$

```

The command `ansible-galaxy init <role_name>` automatically creates a role's own directory with the corresponding configurations, tasks, and defaults folders. It already has the `main.yml` file automatically created so all I needed to do was move the tasks in `site.yml` into their respective `main.yml` files. Running the `site.yml` playbook, with the roles defined instead of the tasks, successfully executed them. This made the playbooks more organized and the clearly defined roles ensured the corresponding tasks are isolated in their own environments.

GITHUB LINK: <https://github.com/owen-s-dotdev/CPE212-Santiago.git>

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Answer the following:

1. What is the importance of creating roles?

Creating roles helps us organize and properly manage the tasks meant for specific managed nodes without interrupting the operations of other nodes. It helps in security and troubleshooting since we can easily pinpoint the points of failures by checking which role failed or contains the issue. Moreover, it makes our coding cleaner since the codes and tasks are properly organized. In this case, the `site.yml` is cleaner and is more modular.

2. What is the importance of managing files?

Managing files ensures organization and order in our systems. Organizing them into roles, tasks, directories, etc. helps in ensuring all of our codes, configurations, resources, and logs can easily be found and tracked during automation. Having them separate also ensures debugging and security checks becomes easier and more efficient since we can easily

pinpoint where the point(s) of failure is/are located.

4. CONCLUSION/ANALYSIS/LEARNINGS

In conclusion, this activity focuses on file management and role creation in Ansible. Here, I was able to use various Ansible built in modules like copy or unarchive to manage files in remote servers. This makes server management and automation easier since we can directly manage files without the need to manually log in each server. Moreover, adding them to playbooks allows automation. Additionally, I created more roles using ansible-galaxy to better manage the role-specific tasks, improve file management, pinpoint faults, and make code more readable. All in all, this activity helped me better understand how roles can be used to improve automation, security, and file management.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.